

Object Oriented Programming In Java

Saurabh Pandey
Associate Professor
Sitare University

Java Input/Output (I/O) Handling

- What is I/O?
- Why do we need it?
- Overview of Java I/O APIs

What is I/O?

- **Input:** Getting data into a program (e.g., from keyboard, file, network)
- **Output:** Sending data out of a program (e.g., to screen, file, printer)
- Java provides rich **I/O classes in [java.io](#)**

Types of I/O in Java

● **Byte Streams**

● Used for binary data (e.g., images, audio)

● Classes: `InputStream`, `OutputStream`

● **Character Streams**

● Used for text data

● Classes: `Reader`, `Writer`

● **Standard I/O**

● Keyboard (`System.in`)

● Monitor/Console (`System.out`, `System.err`)

Standard I/O Example

```
import java.util.Scanner;

public class StdIOExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.println("Hello, " + name + "!");
    }
}
```

Uses **System.in** (input) and **System.out** (output).

Byte Stream Example

```
import java.io.FileInputStream;
import java.io.FileOutputStream;

public class CopyFile {
    public static void main(String[] args) throws Exception {
        FileInputStream in = new FileInputStream("input.txt");
        FileOutputStream out = new FileOutputStream("output.txt");

        int c;
        while ((c = in.read()) != -1) {
            out.write(c);
        }

        in.close();
        out.close();
    }
}
```

Reads bytes from one file and writes to another.

Character Stream Example

```
import java.io.FileReader;
import java.io.FileWriter;

public class CharStreamDemo {
    public static void main(String[] args) throws Exception {
        FileReader fr = new FileReader("input.txt");
        FileWriter fw = new FileWriter("output.txt");

        int c;
        while ((c = fr.read()) != -1) {
            fw.write(c);
        }

        fr.close();
        fw.close();
    }
}
```

More efficient for **text files** than byte streams.

Buffered I/O

Buffer improves performance by reducing disk access

Classes:

- `BufferedReader` / `BufferedWriter`
- `BufferedInputStream` / `BufferedOutputStream`

```
BufferedReader br = new BufferedReader(new FileReader("data.txt"));
String line;
while ((line = br.readLine()) != null) {
    System.out.println(line);
}
br.close();
```


Exception Handling in I/O

I/O operations often throw **checked exceptions**

Must use **try-catch** or **throws**

```
try (FileReader fr = new FileReader("input.txt")) {  
    int ch;  
    while ((ch = fr.read()) != -1) {  
        System.out.print((char) ch);  
    }  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

Summary

- Java I/O = Handling input/output through **streams**
- **Byte streams** → Binary data
- **Character streams** → Text data
- **Buffered streams** → Faster performance
- Always handle **exceptions** in I/O